

Arquitectura, flujo y ejecución del proyecto fibonacci_photonics

Emanuel Orozco Gallego

28 de septiembre de 2026

Resumen

Manual de arquitectura del repositorio que reproduce Reyes-Gómez *et al.*, Phys. Rev. B **81**, 153101 (2010). Describe cómo está organizado el código, cómo circulan los datos desde un YAML hasta una figura, qué dependencias hacen falta y cómo instalar y ejecutar el proyecto de punta a punta. La física está en *Guía conceptual*; el algoritmo TMM, en *Solución numérica e implementación*; el método de ondas planas, en *El método de expansión en ondas planas*.

Índice

1. Qué es este repositorio	3
2. Árbol del proyecto	3
3. Arquitectura de la biblioteca	3
3.1. Módulos y responsabilidades	4
3.2. Objetos que cruzan las fronteras	4
4. Capa de configuración	5
5. Capa de scripts	5
6. Flujo de una corrida	6
7. Tests	7
8. Dependencias	7
8.1. Tiempo de ejecución (Python)	7
8.2. Sistema, para los PDFs	7
8.3. Opcional	8
9. Cómo ejecutar el proyecto	8
9.1. 1. Entorno	8
9.2. 2. Tests (recomendado primero)	8
9.3. 3. Figuras	8
9.4. 4. Documentos PDF	8
9.5. 5. Todo de una vez	9
9.6. Cambiar un parámetro y volver a correr	9
9.7. Limpieza	9
10. Problemas frecuentes	9
11. Dónde está cada documento	9

1. Qué es este repositorio

Es una reimplementación en Python del Brief Report de 2010 con dos métodos independientes: la matriz de transferencia (TMM), que es el método de producción, y la expansión en ondas planas (PWE), que la verifica. No hay servidor web ni interfaz gráfica: el producto son *datos*, *figuras* y *PDFs*, separados por método en `results/tmm/`, `results/pwe/` y `results/comparison/`.

Tres capas, de dentro hacia afuera:

1. **Biblioteca** `src/fibonacci_photonics/`: física y numérica. No sabe de Matplotlib ni de números de figura.
2. **Scripts** `scripts/{tmm,pwe,comparison}/`: leen YAML, llaman a la biblioteca, dibujan y escriben archivos.
3. **Documentos** `docs/`: LaTeX que incrusta las figuras generadas.

Los parámetros científicos no viven en el código: viven en `configs/figure_ON.yaml`. Cambiar un plasma o un ángulo no exige editar Python.

2. Árbol del proyecto

<code>analisis_numerico/</code>	
<code>src/fibonacci_photonics/</code>	biblioteca (layout src)
<code>core/</code>	constantes, materiales, Fibonacci, parametros, bandas
<code>tmm/</code>	matriz de transferencia y barrido
<code>pwe/</code>	ondas planas: Fourier, solver $k(w)$, forma $w(k)$
<code>analysis/</code>	modos, anchos, comparacion, bordes de banda
<code>plotting.py</code>	estilo y guardado de figuras
<code>src/fibonacci_tmm/</code>	alias del nombre antiguo (compatibilidad)
<code>tests/{core,tmm,pwe,analysis,compat}/</code>	pytest
<code>configs/</code>	YAML por figura (física) + base.yaml
<code>configs/pwe/</code>	parametros numericos del PWE
<code>scripts/_common.py</code>	utilidades compartidas
<code>scripts/tmm/</code>	Figs. 1-6 y convergencia con TMM
<code>scripts/pwe/</code>	benchmark del libro, estudios y Figs. 1-6 con PWE
<code>scripts/comparison/</code>	superposiciones, errores, tablas y tiempos
<code>scripts/reproduce_*.py</code>	envoltorios antiguos (llaman a <code>scripts/tmm/</code>)
<code>scripts/verify_results.py</code>	verificacion automatica (make verify)
<code>results/tmm/figure_ON/</code>	data/, output/, README.md
<code>results/pwe/<estudio>/</code>	data/, output/
<code>results/comparison/</code>	figuras, summary.json y tables/*.tex
<code>figures -> results/tmm</code>	enlace de compatibilidad
<code>references/pwe/</code>	PWE-1d.ipynb, PWE-2D.m (el PDF del libro es local)
<code>references/paper/</code>	articulo APS y recortes (local, no se sube)
<code>docs/</code>	auditoria, papers, guias, este documento
<code>presentacion/</code>	presentacion (no la toca make results)
<code>notebooks/</code>	exploracion
<code>pyproject.toml</code>	paquete, pytest, pythonpath
<code>requirements.txt</code>	pip
<code>Makefile</code>	tests, resultados, PDFs

El entorno virtual `.venv/` y las cachés (`__pycache__`, `.pytest_cache`) no forman parte del código fuente. `.gitignore` también excluye auxiliares de LaTeX y el directorio `_paper_pages/` (recortes del PDF original).

3. Arquitectura de la biblioteca

El paquete usa el *src layout*: las importaciones son `from fibonacci_photonics....` y Setuptools busca paquetes en `src/` (`pyproject.toml`). Pytest añade `src` a `PYTHONPATH`. Los scripts hacen `sys.path.insert` a `src/` al arrancar.

El nombre antiguo `fibonacci_tmm` se conserva como alias: cada módulo `fibonacci_tmm.X` reemplaza su entrada en `sys.modules` por el módulo nuevo correspondiente, así que los cuadernos, la presentación y cualquier código externo siguen funcionando sin cambios (`tests/compat/test_legacy_imports.py` lo comprueba).

3.1. Módulos y responsabilidades

Módulo	Depende de	Responsabilidad
<code>core/constants.py</code>	—	c , GHz, mm, tolerancias
<code>core/fibonacci.py</code>	constants	S_m , F_m , L_m
<code>core/materials.py</code>	—	aire y Drude $\varepsilon(\omega)$, $\mu(\omega)$
<code>core/electromagnetics.py</code>	constants	n , q , Q , χ , TE/TM
<code>core/params.py</code>	materials, electromagnetics	YAML $\rightarrow$ SI
<code>core/bands.py</code>	—	DispersionScan , máscara de banda, kL_m/π
<code>core/effective_medium.py</code>	materials	$\langle n \rangle$, gap de índice nulo
<code>core/provenance.py</code>	params, constants	metadatos JSON de cada corrida
<code>tmm/transfer_matrix.py</code>	core	M , T_m , R_m
<code>tmm/dispersion.py</code>	transfer_matrix, bands	barrido TMM
<code>pwe/fourier.py</code>	fibonacci	coeficientes de la celda, Toeplitz
<code>pwe/bloch.py</code>	fourier, params	QEP en k , selección del modo
<code>pwe/dispersion.py</code>	bloch, bands	barrido PWE, SemitraceEvaluator
<code>pwe/eigenfrequency.py</code>	fourier	forma $\omega(k)$ del libro y de Drude
<code>analysis/plasmon_modes.py</code>	bands, fibonacci	recuento de subbandas
<code>analysis/bandwidth.py</code>	plasmon_modes, band_edges	$\nu_{\min}$, $\nu_{\max}$ vs θ
<code>analysis/comparison.py</code>	bands	métricas TMM vs PWE, Brent
<code>analysis/band_edges.py</code>	comparison	localización, refinamiento y validación de subbandas
<code>plotting.py</code>	bands, matplotlib	estilo PRB, curvas, <code>save_figure</code>
<code>__init__.py</code>	los anteriores	API pública

La dependencia es acíclica y va de lo más físico a lo más aplicado: `core` $\rightarrow$ `tmm` y `pwe` (en paralelo, sin depender uno del otro) $\rightarrow$ `analysis` $\rightarrow$ dibujo. Los dos métodos producen el mismo objeto **DispersionScan**, de modo que el análisis (modos, anchos, bordes) y el dibujo no saben qué método generó los datos.

`plotting.py` es el único módulo de la biblioteca que importa Matplotlib. El núcleo numérico puede usarse sin pintar nada.

3.2. Objetos que cruzan las fronteras

- **SuperlatticeSpec**: espesores en metros y Drude en rad/s. Lo construye `spec_from_mapping` a partir del YAML.
- **DispersionScan**: vectores ν , R_m , máscara de banda, kL_m/π . Lo produce `scan_dispersion`.
- **FrequencyInterval**: un trozo conexo $[\nu_{\min}, \nu_{\max}]$.
- **BandwidthPoint**: anchos y centros a un θ dado (Fig. 6).

4. Capa de configuración

Archivo	Uso
<code>configs/base.yaml</code>	$a = b = 12$ mm, aire, c SI
<code>configs/figure_01.yaml</code>	$\nu_e = \nu_m = 3$ GHz, $m = 3, 4$, cuatro θ
<code>configs/figure_02.yaml</code>	zoom 2–4 GHz, $m = 3 \dots 6$
<code>configs/figure_03.yaml</code>	como Fig. 1 con $\nu_m = 1$ GHz
<code>configs/figure_04.yaml</code>	ventanas densas cerca de 1 GHz
<code>configs/figure_05.yaml</code>	palos vs $m = 2 \dots 8$
<code>configs/figure_06.yaml</code>	$\nu(\theta)$ relleno, $m = 2 \dots 7$ (inferida)
<code>configs/pwe/figure_0N.yaml</code>	armónicos por capa, mallas y órdenes del PWE
<code>configs/pwe/book_benchmark.yaml</code>	cristal $\varepsilon_1 = 1$, $\varepsilon_2 = 9$ del libro
<code>configs/pwe/convergence.yaml</code>	estudios de convergencia y costo
<code>configs/pwe/spectral_pollution.yaml</code>	forma $\omega(k)$ con Drude
<code>configs/comparison/efficiency.yaml</code>	estudio de tiempos TMM vs PWE en un núcleo

La física de cada figura está en un solo sitio (`configs/figure_0N.yaml`) y la leen los dos métodos; `configs/pwe/` solo añade parámetros numéricos del PWE.

Cada YAML mezcla *física* (espesores, plasmas, polarización) con *numérica de gráfica* (malla de ν , lista de θ , ventanas). `load_spec` / `spec_from_mapping` solo leen la física. El resto lo consume el script de la figura.

Campo `source`: `paper` si el original da los números; `inferred_from_figures_3_to_5` en la Fig. 6.

5. Capa de scripts

`scripts/_common.py` reúne utilidades compartidas: cargar YAML, crear los directorios `results/<método>/<estudio>/{data,output}`, guardar archivos `.npz` y JSON, escribir el sidecar de provenance y repartir tareas entre procesos (`parallel_map`, con un hilo de BLAS por proceso; `PWE_WORKERS=n` limita su número).

Cada `scripts/tmm/figure_0N.py` hace el mismo esqueleto:

1. `load_figure_config("figure_0N.yaml")`.
2. Construir la malla de ν (y de θ si aplica).
3. Llamar a `scan_dispersion` o a `bandwidth_versus_angle`.
4. Dibujar con `plotting`.
5. Escribir PDF, PNG (300 dpi), SVG, datos y `README.md` en `results/tmm/figure_0N/`.

`scripts/pwe/figure_0N.py` sigue el mismo esqueleto con `fibonacci_photonics.pwe.scan_dispersion` y escribe en `results/pwe/figure_0N/`. Además, `scripts/pwe/` contiene el benchmark del libro (`book_benchmark.py`), la ilustración de la celda (`cell_fourier.py`), la contaminación espectral de la forma $\omega(k)$ (`spectral_pollution.py`) y la convergencia y el costo (`convergence.py`).

`scripts/comparison/` lee los datos de los dos métodos, recalcula la TMM en la misma malla que el PWE y escribe superposiciones, errores, `summary.json` y las tablas LaTeX de `results/comparison/tables/` que incluyen el paper del PWE y el anexo de comparación. `efficiency.py` mide en un núcleo el tiempo por frecuencia frente a m , el diagrama trabajo-precisión, el costo de cada figura y la memoria (`-plot-only` redibuja sin medir).

Cada carpeta tiene un `run_all.py` que lanza sus scripts en orden con el mismo intérprete (`sys.executable`) y aborta si uno falla (`check=True`). Los envoltorios

scripts/reproduce_figure_ON.py y scripts/reproduce_all.py ejecutan los de scripts/tmm/ con runpy.

scripts/tmm/convergence.py no pinta el paper: estudia cómo $\Delta\nu$ de la Fig. 4 cambia con el número de puntos.

6. Flujo de una corrida

El camino feliz, para *una* figura, es:

```
configs/figure_ON.yaml
|
v
spec_from_mapping          (mm, GHz -> SI)
|
v
SuperlatticeSpec
|
+--> tmm.scan_dispersion(spec, m, theta, nu, TE)
|      +--> semitrace_by_recurrence   (por defecto)
+--> pwe.scan_dispersion(spec, m, theta, nu, TE, n_max)
|      +--> QEP en k (regla inversa), R = Re cos(kL)
|      v
|      DispersionScan   (mismo objeto para los dos metodos)
|
+--> detect_plasmon_modes / allowed_intervals   (Figs. 2, 4, 5)
+--> bandwidth_versus_angle                     (Fig. 6)
|
v
plotting.save_figure
|
+--> results/<metodo>/figure_ON/output/figure_ON.{pdf,png,svg}
+--> results/<metodo>/figure_ON/data/*.npz
+--> results/<metodo>/figure_ON/data/summary.json
+--> results/<metodo>/figure_ON/data/metadata.json
```

Los PDFs científicos no recalculan física: incluyen las figuras ya generadas en **results/** (o en **figures/**, que apunta a **results/tmm/**). Por eso hay que generar los resultados *antes* de compilar los documentos si se cambió el código.

Flujo del Makefile:

```
make all
make tests      -> pytest tests
make results    -> make tmm pwe comparison
make tmm        -> scripts/tmm/run_all.py (Figs. 1-6 + convergencia)
make pwe        -> scripts/pwe/run_all.py (horas de CPU)
make comparison -> scripts/comparison/run_all.py (incluye eficiencia)
make documents  -> pwe-docs, guides, paper, docs
make pwe-docs   -> metodo_ondas_planas, comparacion_tmm_pwe
make guides     -> guia_conceptual, solucion_numerica, arquitectura
make paper      -> paper_reimplementation
make docs       -> software_documentation
make verify     -> scripts/verify_results.py
make reproduce  -> results + documents + verify (sin tests)
make figures    -> alias de make tmm
make presentation -> presentacion/ (objetivo aparte, no lo llama make all)
```

Todos los PDF se compilan con una sola regla de patrón (dos pases de **pdflatex**, que se detiene en el primer error). Cada PDF depende de su **.tex** y de las figuras y tablas que incluye, así que **make** solo recompila lo que cambió. El PDF se copia también a la raíz del repo (esas copias no se suben) para abrirlo sin entrar en docs/.

`make verify` comprueba que existan todas las salidas, que se cumplan los criterios de acuerdo TMM vs PWE (conteos F_{m-2} , errores de semitraza, de borde y de ancho), que cada tabla y figura citada exista y que cada PDF esté al día y compile sin errores ni referencias indefinidas. Con `-reference DIR` compara además los datos con una corrida anterior.

7. Tests

`pyproject.toml` configura `testpaths = ["tests"]` y `pythonpath = ["src"]`. No hace falta instalar el paquete para testear.

Archivo	Qué garantiza
<code>core/test_fibonacci.py</code>	palabras, inflación, N_B , L_m
<code>core/test_materials.py</code>	ceros de Drude, ν_e , ν_m
<code>core/test_params.py</code>	YAML de las seis figuras
<code>tmm/test_transfer_matrix.py</code>	$\det T = 1$, eqs. (7)–(9), dos vías de R_m
<code>tmm/test_dispersion.py</code>	máscara de banda, gap $\langle n \rangle = 0$
<code>pwe/test_fourier.py</code>	ec. (4.38), fracción de llenado, Toeplitz, síntesis
<code>pwe/test_solver.py</code>	libro, $R_{\text{PWE}} \approx R_{\text{TMM}}$, Laurent vs inversa, contaminación
<code>analysis/test_validation.py</code>	$N_{\text{modos}} = F_{m-2}$
<code>analysis/test_comparison.py</code>	métricas, Brent, localización y validación de bandas
<code>compat/test_legacy_imports.py</code>	el alias <code>fibonacci_tmm</code>

`conftest.py` fija un hilo de BLAS (las matrices del PWE son pequeñas y el multihilo solo añade sobrecarga) y expone dos fixtures de `SuperlatticeSpec`: plasmas iguales (3,3 GHz, Figs. 1–2) y partidos (3,1 GHz, Figs. 3–6).

Tres casos se omiten (`skip`) cuando una identidad de Fibonacci pide $k > m$. El resto debe pasar.

8. Dependencias

8.1. Tiempo de ejecución (Python)

Declaradas en `requirements.txt` y en `pyproject.toml`. Python ≥ 3.10 (`requires-python`). El desarrollo se hizo con Python 3.14.

Paquete	Mínimo	Para qué
<code>numpy</code>	1.24	arrays, $\sqrt{}$, trigonométricas complejas
<code>scipy</code>	1.10	Brent, <code>eigh</code> generalizado del PWE
<code>matplotlib</code>	3.7	Figs. 1–6
<code>pyyaml</code>	6.0	configs
<code>pytest</code>	7.0	tests (extra <code>dev</code> en <code>pyproject</code>)

No hay CUDA, MPI ni bases de datos. No se descarga nada en tiempo de cálculo: el paper original es un PDF local.

8.2. Sistema, para los PDFs

Además de Python:

- `make`
- `pdflatex` (TeX Live). Estilos usados: `article`, `babel-spanish`, `amsmath`, `graphicx`, `booktabs`, `hyperref`, `listings`, `geometry`, `siunitx`, `enumitem`.

Sin LaTeX se pueden igual ejecutar tests y figuras; fallarán solo `make paper`, `make docs` y `make guides`.

En Arch Linux, un conjunto suficiente es el metapaquete de TeX Live más los estilos anteriores. En Debian/Ubuntu suele bastar `texlive-latex-extra` y `texlive-lang-spanish`.

8.3. Opcional

- Instalar el paquete en modo editable: `pip install -e ".[dev]"` (entonces no hace falta el `sys.path` de los scripts, aunque ellos lo siguen insertando).
- Git: `provenance.py` intenta guardar el hash de HEAD en el JSON de cada figura; si no hay repo, el campo queda vacío.

9. Cómo ejecutar el proyecto

Sustituya `analisis_numerico` por la ruta real del clon.

9.1. 1. Entorno

```
cd analisis_numerico
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Compruebe `python3 --version ≥ 3.10`. Si `python3` no existe, use el intérprete concreto (por ejemplo `python3.12`).

9.2. 2. Tests (recomendado primero)

```
.venv/bin/pytest
# equivalente:
make tests
```

Debe terminar con todos los casos en verde salvo tres `skipped`.

9.3. 3. Figuras

```
.venv/bin/python scripts/tmm/run_all.py          # TMM, segundos
.venv/bin/python scripts/tmm/figure_04.py        # una sola
PWE_WORKERS=8 .venv/bin/python scripts/pwe/run_all.py # PWE, horas
.venv/bin/python scripts/comparison/run_all.py    # requiere los dos
make results  # los tres
```

Salida esperada: bloques “Figura N escrita...” y archivos en `results/<método>/figure_0N/output/`.

9.4. 4. Documentos PDF

```
make paper      # paper_reimplementation.pdf
make docs       # software_documentation.pdf
make guides     # guia_conceptual.pdf, solucion_numerica.pdf,
                # arquitectura.pdf
make pwe-docs   # metodo_ondas_planas.pdf, comparacion_tmm_pwe.pdf
make documents  # todos
make verify     # comprobacion_final
```

`make guides` exige que las figuras TMM existan, y `make pwe-docs` que existan `results/pwe/` y `results/comparison/`.

9.5. 5. Todo de una vez

```
source .venv/bin/activate
make all
```

Orden interno: tests → resultados (TMM, PWE, comparación) → documentos → verificación. Si los tests fallan, make se detiene y no genera figuras.

9.6. Cambiar un parámetro y volver a correr

Edite el YAML (por ejemplo `omega_m_over_2pi_ghz` en `configs/figure_03.yaml`) y ejecute solo el script de esa figura. No recompile el paper si no necesita el PDF actualizado.

Para TM: ponga `polarization: TM` en el YAML. El original no grafica TM; el código sí la implementa ($\chi = \varepsilon$).

9.7. Limpieza

```
make clean    # auxiliares de LaTeX de docs/ y presentacion/
```

No borra figuras ni el entorno virtual.

10. Problemas frecuentes

- `ModuleNotFoundError: fibonacci_photonics`: está ejecutando un script fuera de `scripts/` sin `src` en el path. Use los scripts del repo o `PYTHONPATH=src`.
- `ZeroDivisionError` en $\omega = 0$: el Drude diverge; el YAML no debe incluir el origen en el barrido (la Fig. 1 empieza en 0.15 GHz).
- Fig. 1 sin gaps a $\theta = 0$ y $\omega_e = \omega_m$: no es un bug, es adaptación de impedancia.
- `pdflatex: command not found`: instale TeX Live; tests y figuras no lo necesitan.
- Figuras viejas en el paper: regenerar con `scripts/tmm/run_all.py` y luego `make paper`.
- El PWE tarda mucho o satura la máquina: limite los procesos con `PWE_WORKERS=n`.
- El Makefile apunta a `.venv/bin/python`. Si el venv está en otra ruta, invoque los comandos con el intérprete explícito.

11. Dónde está cada documento

PDF en la raíz	Contenido
<code>paper_reimplementation.pdf</code>	artículo de la reproducción
<code>software_documentation.pdf</code>	manual corto de la API
<code>guia_conceptual.pdf</code>	física desde cero
<code>solucion_numerica.pdf</code>	TMM y código de las figuras
<code>arquitectura.pdf</code>	este documento
<code>metodo_ondas_planas.pdf</code>	formulación y resultados del PWE
<code>comparacion_tmm_pwe.pdf</code>	anexo de comparación TMM vs PWE

Fuentes LaTeX en `docs/`. Auditoría previa al código: `docs/fase_A_auditoria_matematica.md`. Informe de validación: `docs/INFORME_VALIDACION.md`. Paper original: PDF en la raíz del repo (no versionado). Capítulo del libro y cuaderno del PWE: `references/pwe/`.

12. Resumen operativo

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest
python scripts/tmm/run_all.py
make paper docs guides
```

Eso deja tests en verde, las seis figuras y todos los PDF en la raíz del proyecto.